

Лабораторная работа №14.
Программирование в командном
процессоре ОС UNIX. Расширенное
программирование

Дисциплина: Архитектура компьютеров и операционные системы

Игнатенко Денис Бенъяминович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Подготовка рабочего каталога	8
4.2	Реализация упрощённого механизма семафоров	8
4.3	Реализация аналога команды <code>map</code>	13
4.4	Генерация случайной последовательности букв	16
5	Ответы на контрольные вопросы	19
5.1	1. Найдите синтаксическую ошибку в строке <code>while [\$1 != "exit"]</code>	19
5.2	2. Как объединить несколько строк в одну?	19
5.3	3. Найдите информацию об утилите <code>seq</code> . Какими иными способами можно реализовать её функционал при программировании на <code>bash</code> ?	20
5.4	4. Какой результат даст вычисление выражения <code>\$((10/3))</code> ?	20
5.5	5. Укажите кратко основные отличия командной оболочки <code>zsh</code> от <code>bash</code>	21
5.6	6. Проверьте, верен ли синтаксис конструкции <code>for ((a=1; a <= LIMIT; a++))</code>	21
5.7	7. Сравните язык <code>bash</code> с другими языками программирования. Какие преимущества и недостатки у <code>bash</code> ?	21
6	Выводы	22
	Список литературы	23

Список иллюстраций

4.1	Создание каталога lab14 и проверка текущего пути	8
4.2	Проверка прав доступа файла semaphore.sh	9
4.3	Запуск semaphore.sh для одного процесса	11
4.4	Работа semaphore.sh для двух процессов	12
4.5	Работа semaphore.sh для трёх процессов	13
4.6	Просмотр содержимого каталога /usr/share/man/man1	14
4.7	Проверка прав доступа файла myman.sh	14
4.8	Открытие справочной страницы для команды ls	15
4.9	Сообщение об отсутствии справки для команды sdlfkjhdsalhfa	16
4.10	Проверка прав доступа файла random_letters.sh	16
4.11	Генерация случайных последовательностей разной длины	17
4.12	Сообщение об использовании random_letters.sh	18

Список таблиц

3.1	Реализованные командные файлы	7
-----	---	---

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX и освоить написание более сложных командных файлов с использованием логических управляющих конструкций, циклов и аргументов командной строки.

2 Задание

- Написать командный файл, реализующий упрощённый механизм семафоров.
- Реализовать аналог команды `map` с помощью командного файла.
- Разработать командный файл, генерирующий случайную последовательность букв латинского алфавита с использованием переменной `$RANDOM`.

3 Теоретическое введение

Командные файлы `bash` удобны для автоматизации типовых действий в UNIX-подобных системах. Они позволяют организовывать ветвления, циклы, обработку аргументов командной строки и взаимодействие с системными утилитами.

В лабораторной работе были реализованы три типовых сценария: синхронизация доступа к ресурсу, обращение к справочной подсистеме и генерация случайных данных. Их краткая характеристика приведена в таблице 1.

Таблица 3.1: Реализованные командные файлы

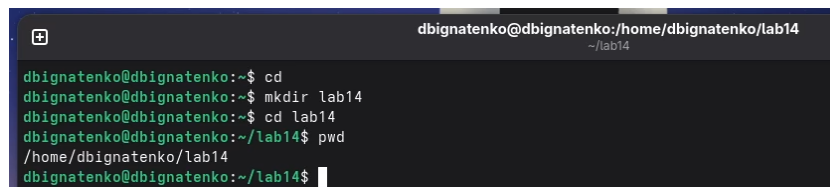
Файл	Назначение	Основная идея реализации
<code>semaphore.sh</code>	Упрощённый семафор	Захват ресурса через атомарное создание каталога <code>resource.lock</code>
<code>myman.sh</code>	Аналог <code>man</code>	Поиск архива справки в <code>/usr/share/man/man1</code> и открытие найденного файла
<code>random_letters.sh</code>	Генерация случайной строки	Преобразование значения <code>\$RANDOM</code> в диапазон кодов букв <code>a-z</code>

4 Выполнение лабораторной работы

4.1 Подготовка рабочего каталога

В начале работы был создан каталог lab14, после чего выполнен переход в него и проверен текущий путь командой pwd (рис. 1).

```
cd
mkdir lab14
cd lab14
pwd
```



```
dbignatenko@dbignatenko:~/home/dbignatenko/lab14
~ /lab14

dbignatenko@dbignatenko:~$ cd
dbignatenko@dbignatenko:~$ mkdir lab14
dbignatenko@dbignatenko:~$ cd lab14
dbignatenko@dbignatenko:~/lab14$ pwd
/home/dbignatenko/lab14
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.1: Создание каталога lab14 и проверка текущего пути

4.2 Реализация упрощённого механизма семафоров

Сначала был создан и сделан исполняемым файл semaphore.sh (рис. 2).

```
nano semaphore.sh
chmod +x semaphore.sh
ls -l semaphore.sh
```



```
dbignatenko@dbignatenko:/home/dbignatenko/lab14
~ /lab14
dbignatenko@dbignatenko:~/lab14$ nano semaphore.sh
dbignatenko@dbignatenko:~/lab14$ chmod +x semaphore.sh
dbignatenko@dbignatenko:~/lab14$ ls -l semaphore.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 829 апр 28 21:23 semaphore.sh
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.2: Проверка прав доступа файла semaphore.sh

Листинг скрипта:

```
#!/bin/bash

LOCKDIR="resource.lock"

T1=15
T2=10

PROCESS_NAME=$1

if [ -z "$PROCESS_NAME" ]; then
    PROCESS_NAME="process_$$"
fi

echo "$PROCESS_NAME запущен"

WAITED=0

while ! mkdir "$LOCKDIR" 2>/dev/null; do
    echo "$PROCESS_NAME: ресурс занят, ожидание..."
    sleep 1
    WAITED=$((WAITED + 1))
```

```

    if [ "$WAITED" -ge "$T1" ]; then
        echo "$PROCESS_NAME: время ожидания истекло, ресурс не получен"
        exit 1
    fi
done

trap 'rm -rf "$LOCKDIR"' EXIT

echo "$PROCESS_NAME: ресурс свободен, занимаю ресурс"

for ((i=1; i<=T2; i++)); do
    echo "$PROCESS_NAME: использует ресурс... $i сек."
    sleep 1
done

echo "$PROCESS_NAME: освобождает ресурс"
rm -rf "$LOCKDIR"

trap - EXIT

echo "$PROCESS_NAME завершён"

```

При одиночном запуске процесс сразу получает доступ к ресурсу, удерживает его в течение десяти секунд и затем освобождает (рис. 3).

```
./semaphore.sh process1
```

```
dbignatenko@dbignatenko:~/lab14$ ./semaphore.sh process1
process1 запущен
process1: ресурс свободен, занимаю ресурс
process1: использует ресурс... 1 сек.
process1: использует ресурс... 2 сек.
process1: использует ресурс... 3 сек.
process1: использует ресурс... 4 сек.
process1: использует ресурс... 5 сек.
process1: использует ресурс... 6 сек.
process1: использует ресурс... 7 сек.
process1: использует ресурс... 8 сек.
process1: использует ресурс... 9 сек.
process1: использует ресурс... 10 сек.
process1: освобождает ресурс
process1 завершён
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.3: Запуск semaphore.sh для одного процесса

При одновременном запуске двух процессов один из них занимает ресурс, а второй переходит в режим ожидания до освобождения блокировки (рис. 4). По фактическому выводу на скриншоте первым ресурс захватывает process2, а process1 ожидает.

```
./semaphore.sh process1 &
```

```
./semaphore.sh process2
```

```
dbignatenko@dbignatenko:~/home/dbignatenko/lab14$ ./semaphore.sh process1 & ./semaphore.sh process2
[1] 4792
process2 запущен
process1 запущен
process2: ресурс свободен, занимаю ресурс
process2: использует ресурс... 1 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 2 сек.
process1: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process2: использует ресурс... 3 сек.
process2: использует ресурс... 4 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 5 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 6 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 7 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 8 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 9 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 10 сек.
process1: ресурс занят, ожидание...
process2: освобождает ресурс
process2 завершён
process1: ресурс свободен, занимаю ресурс
process1: использует ресурс... 1 сек.
dbignatenko@dbignatenko:~/lab14$ process1: использует ресурс... 2 сек.
process1: использует ресурс... 3 сек.
process1: использует ресурс... 4 сек.
process1: использует ресурс... 5 сек.
process1: использует ресурс... 6 сек.
process1: использует ресурс... 7 сек.
process1: использует ресурс... 8 сек.
process1: использует ресурс... 9 сек.
process1: использует ресурс... 10 сек.
process1: освобождает ресурс
process1 завершён
```

```
dbignatenko@dbignatenko:~/lab14$ rm -rf resource.lock
dbignatenko@dbignatenko:~/lab14$ ./semaphore.sh process1 &
./semaphore.sh process2 &
./semaphore.sh process3
[1] 6116
[2] 6117
process3 запущен
process2 запущен
process1 запущен
process3: ресурс занят, ожидание...
process2: ресурс свободен, занимаю ресурс
process2: использует ресурс... 1 сек.
process1: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process2: использует ресурс... 2 сек.
process3: ресурс занят, ожидание...
process2: использует ресурс... 3 сек.
process1: ресурс занят, ожидание...
process3: ресурс занят, ожидание...
process2: использует ресурс... 4 сек.
process1: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process2: использует ресурс... 5 сек.
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process2: использует ресурс... 6 сек.
process1: ресурс занят, ожидание...
process2: использует ресурс... 7 сек.
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
process2: использует ресурс... 8 сек.
process3: ресурс занят, ожидание...
process1: ресурс занят, ожидание...
```

Рис. 4.5: Работа semaphore.sh для трёх процессов

Таким образом, каталог `resource.lock` используется как простая блокировка, а цикл ожидания обеспечивает последовательный доступ нескольких процессов к одному ресурсу.

4.3 Реализация аналога команды `man`

Перед созданием скрипта был просмотрен каталог `/usr/share/man/man1`, содержащий архивы справочных страниц для пользовательских команд (рис. 6).

```
ls /usr/share/man/man1 | head
```

```
dbignatenko@dbignatenko:~/lab14$ ls /usr/share/man/man1 | head
.:1.gz
[.:1.gz
a2ping.1.gz
ab.1.gz
abrt.1.gz
abrt-action-analyze-backtrace.1.gz
abrt-action-analyze-c.1.gz
abrt-action-analyze-cpp-local.1.gz
abrt-action-analyze-oops.1.gz
abrt-action-analyze-python.1.gz
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.6: Просмотр содержимого каталога /usr/share/man/man1

После этого был создан и сделан исполняемым файл `myman.sh` (рис. 7).

```
chmod +x myman.sh
```

```
ls -l myman.sh
```

```
dbignatenko@dbignatenko:~/lab14$ chmod +x myman.sh
dbignatenko@dbignatenko:~/lab14$ ls -l myman.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 326 апр 28 21:39 myman.sh
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.7: Проверка прав доступа файла `myman.sh`

Листинг скрипта:

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Использование: $0 имя_команды"
    exit 1
fi

COMMAND=$1
MAN_DIR="/usr/share/man/man1"
FILE="$MAN_DIR/$COMMAND.1.gz"
```

```

if [ -f "$FILE" ]; then
    less "$FILE"
else
    echo "Справка для команды '$COMMAND' не найдена в $MAN_DIR"
fi

```

При вызове с существующей командой `ls` скрипт открывает содержимое справочной страницы в просмотрщике `less` (рис. 8).

```
./myman.sh ls
```

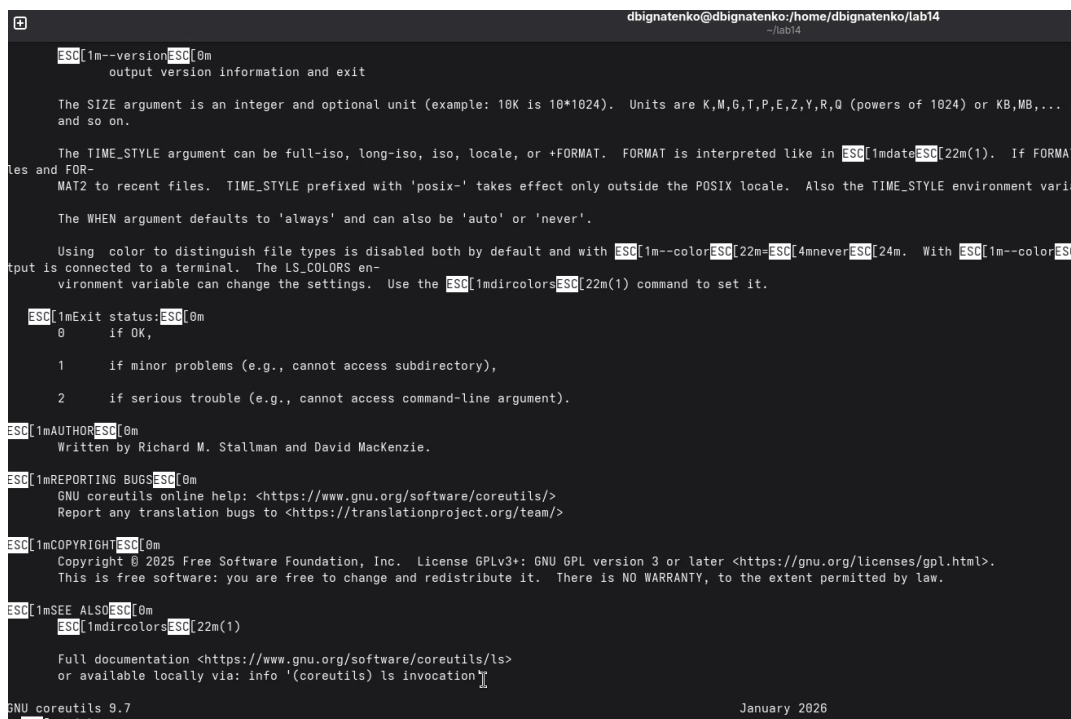


Рис. 4.8: Открытие справочной страницы для команды `ls`

При обращении к несуществующей команде `sdlfkjhjdaslhfa` выводится сообщение об отсутствии файла справки в каталоге `man1` (рис. 9).

```
./myman.sh sdlfkjhjdaslhfa
```

```
dbignatenko@dbignatenko:~/lab14$ ./myman.sh sdlfkjhdsalhfa
Справка для команды 'sdlfkjhdsalhfa' не найдена в /usr/share/man/man1
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.9: Сообщение об отсутствии справки для команды sdlfkjhdsalhfa

Скрипт корректно обрабатывает оба основных сценария: наличие и отсутствие соответствующего man-файла.

4.4 Генерация случайной последовательности букв

Для третьего задания был подготовлен и сделан исполняемым файл random_letters.sh (рис. 10).

```
chmod +x random_letters.sh
```

```
ls -l random_letters.sh
```

```
dbignatenko@dbignatenko:~/lab14$ chmod +x random_letters.sh
dbignatenko@dbignatenko:~/lab14$ ls -l random_letters.sh
-rwxr-xr-x. 1 dbignatenko dbignatenko 381 апр 28 21:44 random_letters.sh
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.10: Проверка прав доступа файла random_letters.sh

Листинг скрипта:

```
#!/bin/bash
```

```
if [ -z "$1" ]; then
    echo "Использование: ./random_letters.sh длина_строки"
    exit 1
fi
```

```
LENGTH=$1
```

```
RESULT=""
```



```

for ((i=1; i<=LENGTH; i++)); do
    NUM=$((RANDOM % 26))
    ASCII=$((97 + NUM))
    LETTER=$(printf "\\$(printf '%03o' "$ASCII")")
    RESULT="${RESULT}${LETTER}"
done

echo "Случайная последовательность:"
echo "$RESULT"

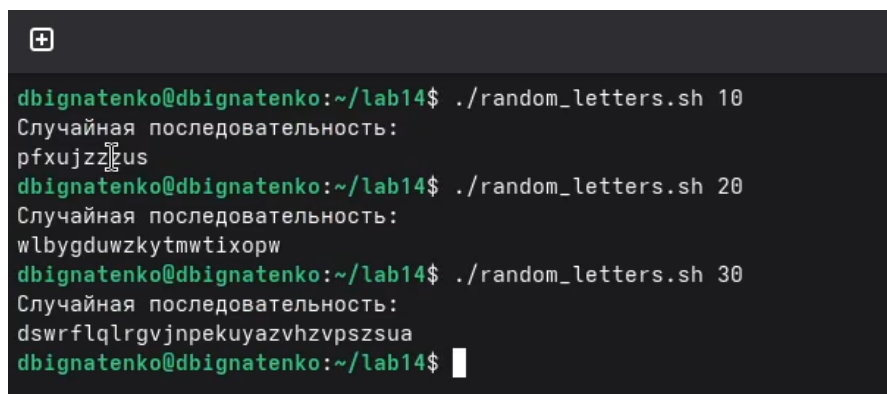
```

На скриншоте видно три запуска программы с длинами 10, 20 и 30, в результате которых формируются различные строки из строчных латинских букв (рис. 11).

```

./random_letters.sh 10
./random_letters.sh 20
./random_letters.sh 30

```



```

dbignatenko@dbignatenko:~/lab14$ ./random_letters.sh 10
Случайная последовательность:
pfxujzzfEus
dbignatenko@dbignatenko:~/lab14$ ./random_letters.sh 20
Случайная последовательность:
wlbygdwzkytmwtixorw
dbignatenko@dbignatenko:~/lab14$ ./random_letters.sh 30
Случайная последовательность:
dswrflqlrgvjnpkuyazvhzvpssua
dbignatenko@dbignatenko:~/lab14$ █

```

Рис. 4.11: Генерация случайных последовательностей разной длины

При запуске без аргумента скрипт выводит подсказку по использованию, что позволяет корректно обработать ошибочный вызов (рис. 12).

```

./random_letters.sh

```

```
dbignatenko@dbignatenko:~/lab14$ ./random_letters.sh
Использование: ./random_letters.sh длина_строки
dbignatenko@dbignatenko:~/lab14$
```

Рис. 4.12: Сообщение об использовании random_letters.sh

Реализация использует переменную \$RANDOM, приводит её значение к диапазону от 0 до 25, а затем преобразует его в ASCII-код строчной латинской буквы.

5 Ответы на контрольные вопросы

5.1 1. Найдите синтаксическую ошибку в строке `while` `[$1 != "exit"]`

Ошибка состоит в отсутствии пробелов после `[` и перед `]`. Корректная запись:

```
while [ "$1" != "exit" ]
```

Дополнительно переменную лучше заключать в кавычки, чтобы избежать ошибок при пустом значении.

5.2 2. Как объединить несколько строк в одну?

В `bash` строки можно объединять через подстановку переменных:

```
a="Hello"  
b="World"  
c="$a $b"  
echo "$c"
```

Также можно постепенно дописывать фрагменты в результирующую строку:

```
result=""  
result="${result}abc"  
result="${result}def"  
echo "$result"
```

5.3 3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash?

Утилита seq выводит последовательность чисел в заданном диапазоне. Например:

```
seq 1 5
```

Аналогичное поведение можно получить с помощью цикла for:

```
for ((i=1; i<=5; i++)); do
    echo "$i"
done
```

или цикла while:

```
i=1
while [ "$i" -le 5 ]; do
    echo "$i"
    i=$((i + 1))
done
```

5.4 4. Какой результат даст вычисление выражения $\$(10/3)$?

Результатом будет число 3, так как в арифметических выражениях bash деление целочисленное.

5.5 5. Укажите кратко основные отличия командной оболочки zsh от bash

zsh обладает более развитыми средствами интерактивной работы: автодополнением, настройкой приглашения, темами и расширенной историей команд. bash более распространён как стандартная оболочка и лучше подходит для переносимых скриптов.

5.6 6. Проверьте, верен ли синтаксис конструкции for ((a=1; a <= LIMIT; a++))

Такую запись нельзя считать полным циклом, так как в ней отсутствуют ключевые слова `do` и `done`. Корректный вариант:

```
LIMIT=10

for ((a=1; a <= LIMIT; a++)); do
    echo "$a"
done
```

5.7 7. Сравните язык bash с другими языками программирования. Какие преимущества и недостатки у bash?

bash удобен для автоматизации системных действий, запуска утилит, работы с файлами, каталогами и процессами. Его основное преимущество состоит в тесной интеграции с UNIX-средой и отсутствии этапа компиляции.

К недостаткам относятся слабая типизация, неудобство отладки крупных программ, ограниченные средства для сложных структур данных и меньшая пригодность для больших проектов по сравнению с языками общего назначения, например Python, C++ или Java.

6 Выводы

В ходе лабораторной работы были изучены приёмы расширенного программирования в командной оболочке UNIX. Были реализованы три командных файла: упрощённый семафор для синхронизации процессов, аналог команды `map` для поиска справки и генератор случайных строк на основе переменной `$RANDOM`. По результатам тестов на скриншотах можно сделать вывод, что все три сценария были успешно реализованы и корректно отрабатывают как штатные, так и ошибочные случаи запуска.

Список литературы